

The Outreach Engine

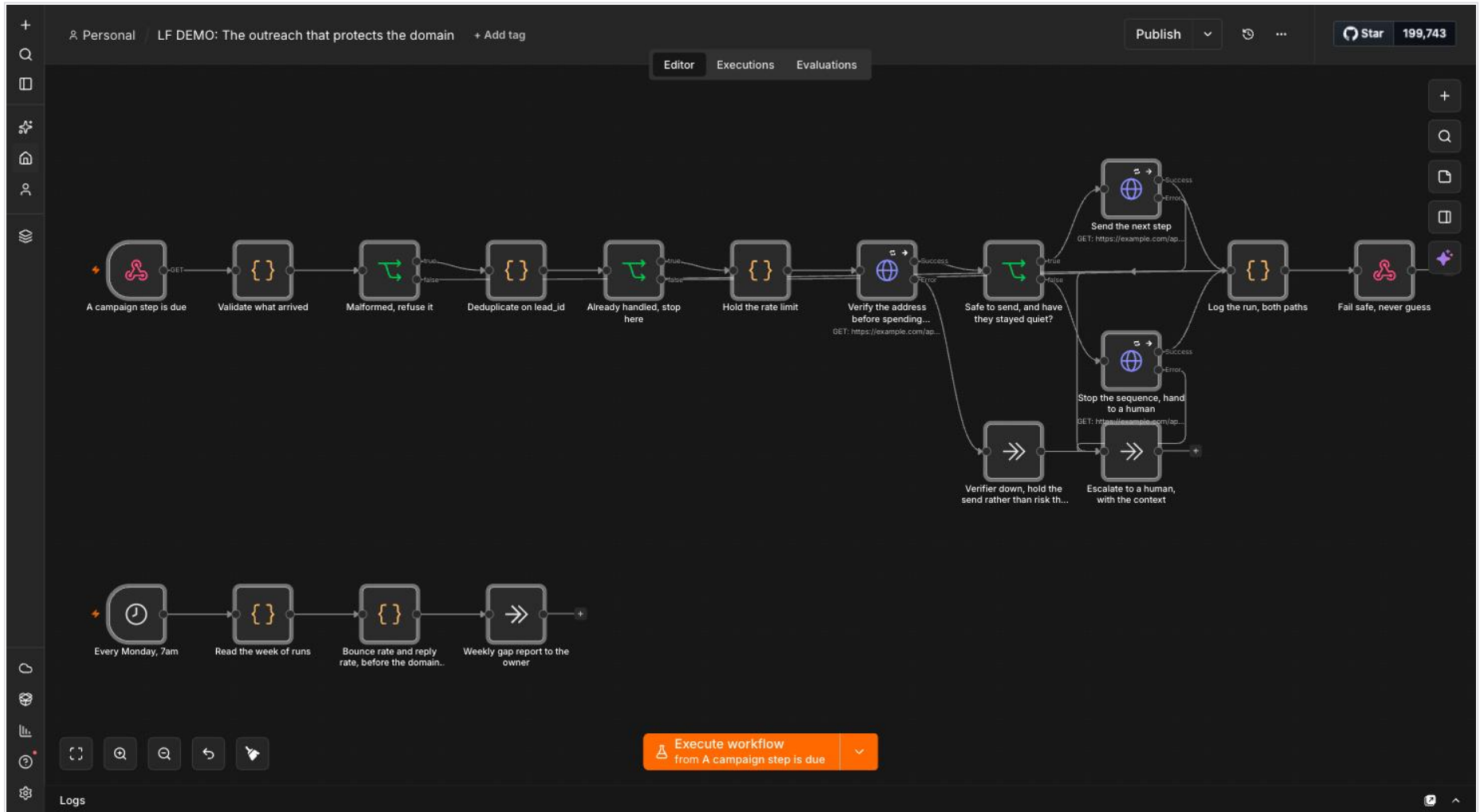
cold-outreach · built in n8n

Volume isn't the problem. Deliverability and reply handling are, and they're what nobody sets up properly.

What it connects to: The sequencer owns the send and the CRM owns the truth about whether that person is already a customer. Emailing an existing client is the mistake that costs the account.

1 of 3 SYSTEM

Here's the sending architecture — validated list, deduplicated, rate-limited to protect the domain.



2 of 3 THINKING

The reply gate stops the sequence the moment someone answers.

Safe to send, and have they stayed quiet?

INPUT

→|

No input data

Execute previous nodes

to view input data

ParametersSettings

Execute step

Conditions

fx

{{ \$json.deliverable === true && \$json.replied !== true && \$json.step <= 3 }}

is true

+ Add condition

☐ Convert types where required

Options

+ Add option

3 of 3 MORE

And the report: what's landing, what's bouncing, what replied.

Bounce rate and reply rate, before the domain pays for it

INPUT

→|

No input data

Execute previous nodes

to view input data

ParametersSettings

Execute step

Mode

Run Once for All Items

Language

JavaScript

JavaScript

CodeAsk AI

```
1 // Bounce rate and reply rate, before the domain pays for it.  
2 const sends = Object.values($getWorkflowStaticData('global').sends || {});  
3 const week = sends.filter(s => s.at > Date.now() - 7*864e5);  
4 const bounced = week.filter(s => s.bounced);  
5 const replied = week.filter(s => s.replied);  
6 const bounceRate = week.length ? (bounced.length / week.length * 100) : 0;  
7 return [{ json: {  
8   sent: week.length,  
9   bounceRate: Math.round(bounceRate * 10) / 10,  
10  replyRate: week.length ? Math.round(replied.length / week.length * 1000) / 10 : 0,  
11  note: bounceRate > 3  
12    ? `Bounce rate ${bounceRate.toFixed(1)}% – above 3% the domain starts paying for it. Pause and reverify the list.`  
13    : 'Bounce rate is inside a safe band. Keep the pace.',  
14 } }];
```

Type \$ for a list of **special vars/methods**. Debug by using console.log() statements and viewing their output in the browser console.